Check for updates

ACM DIGITAL LIBRARY | Association for Computing Machinery | acm open

Latest updates: https://dl.acm.org/doi/10.1145/3691620.3696020

RESEARCH-ARTICLE

MR-Adopt: Automatic Deduction of Input Transformation Function for Metamorphic Testing

CONGYING XU, Hong Kong University of Science and Technology, Hong Kong, Hong Kong

SONGQIANG CHEN, Hong Kong University of Science and Technology, Hong Kong, Hong Kong

JIARONG WU, Hong Kong University of Science and Technology, Hong Kong, Hong Kong

SHING-CHI CHEUNG, Hong Kong University of Science and Technology, Hong Kong, Hong Kong

VALERIO TERRAGNI, University of Auckland, Auckland, AUK, New Zealand

HENGCHENG ZHU, Hong Kong University of Science and Technology, Hong Kong, Hong Kong

View all

Open Access Support provided by:

Hong Kong University of Science and Technology

University of Auckland

PDF Download
3691620.3696020.pdf
30 March 2026
Total Citations: 6
Total Downloads: 1021

Published: 27 October 2024

Citation in BibTeX format

ASE '24: 39th IEEE/ACM International Conference on Automated Software Engineering
October 27 - November 1, 2024
CA, Sacramento, USA

Conference Sponsors:
SIGAI
SIGSOFT

MR-Adopt: Automatic Deduction of Input Transformation Function for Metamorphic Testing

Congying Xu[a,b], Songqiang Chen[a,b], Jiarong Wu[a,b], Shing-Chi Cheung[a,b*], Valerio Terragni[c], Hengcheng Zhu[a,b], Jialun Cao[a,b*]
[a]The Hong Kong University of Science and Technology, Hong Kong
[b]Guangzhou HKUST Fok Ying Tung Research Institute, China
[c]The University of Auckland, New Zealand

ABSTRACT

While a recent study reveals that many developer-written test cases can encode a reusable Metamorphic Relation (MR), over 70% of them directly hard-code the source input and follow-up input in the encoded relation. Such encoded MRs, which do not contain an explicit input transformation to transform the source inputs to corresponding follow-up inputs, cannot be reused with new source inputs to enhance test adequacy.

In this paper, we propose MR-Adopt (*Automatic Deduction Of inPut Transformation*) to automatically deduce the input transformation from the hard-coded source and follow-up inputs, aiming to enable the encoded MRs to be reused with new source inputs. With typically only one pair of source and follow-up inputs available in an MR-encoded test case as the example, we leveraged LLMs to understand the intention of the test case and generate additional examples of source-followup input pairs. This helps to guide the generation of input transformations generalizable to multiple source inputs. Besides, to mitigate the issue that LLMs generate erroneous code, we refine LLM-generated transformations by removing MR-irrelevant code elements with data-flow analysis. Finally, we assess candidate transformations based on encoded output relations and select the best transformation as the result. Evaluation results show that MR-Adopt can generate input transformations applicable to all experimental source inputs for 72.00% of encoded MRs, which is 33.33% more than using vanilla GPT-3.5. By incorporating MR-Adopt-generated input transformations, encoded MR-based test cases can effectively enhance the test adequacy, increasing the line coverage and mutation score by 10.62% and 18.91%, respectively.

CCS CONCEPTS

• **Software and its engineering → Software testing and debugging**.

KEYWORDS

Software Testing, Metamorphic Testing, Metamorphic Relation, Input Transformation, Code Generation, Large Language Models

ACM Reference Format:
Congying Xu, Songqiang Chen, Jiarong Wu, Shing-Chi Cheung, Valerio Terragni, Hengcheng Zhu, and Jialun Cao. 2024. MR-Adopt: Automatic Deduction of Input Transformation Function for Metamorphic Testing. In *39th IEEE/ACM International Conference on Automated Software Engineering (ASE '24), October 27-November 1, 2024, Sacramento, CA, USA*. ACM, New York, NY, USA, 13 pages. https://doi.org/10.1145/3691620.3696020

*Shing-Chi Cheung and Jialun Cao are corresponding authors. Emails: scc@cse.ust.hk, jcaoap@cse.ust.hk.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
ASE '24, October 27-November 1, 2024, Sacramento, CA, USA
© 2024 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 979-8-4007-1248-7/24/10…$15.00
https://doi.org/10.1145/3691620.3696020

1 INTRODUCTION

Metamorphic Testing (MT) is a powerful testing technique to address both the test case generation and the oracle problem [10, 47]. Instead of assessing the outputs of individual inputs, MT validates the behavior of a subject under test (SUT) against Metamorphic Relations (MRs) for the SUT. Each MR defines an *input relation* over a set of related inputs and an *output relation* over the expected outputs for those inputs. One *outstanding benefit* of MT is that once an MR is identified, MT can leverage a wide range of automatically generated inputs (known as *source inputs*) to exercise diverse program behaviors with no need to prepare oracles for individual inputs [65]. MT has achieved success in detecting faults for various software, such as compilers [26, 52], databases [31, 36], machine translation services [5, 64], and question answering systems [63, 69].

Identifying appropriate MRs for a SUT is essential to applying MT. Some studies have focused on MR identification. Earlier approaches either suffer from being labor-intensive and specific to certain domains or pre-defined MR patterns [53, 72, 73] or produce overly generic MRs that are ineffective for testing, as well as recent LLM-based techniques [51, 57]. Recently, Xu et al. [65] report that developers often encode domain knowledge in test cases that exercise MRs. These encoded MRs can be generalized to many new inputs and serve as oracles for more exhaustive testing of the original programs (or programs with similar functionalities), by integrating with automatic input generation techniques [10, 47, 65].

However, Xu et al. [65] show that over 70% of 11,000 MR-encoded test cases (MTCs) in their dataset do not contain explicit input relations. Instead, developers often hard-code the source and follow-up inputs. Figure 1a shows an MR-encoded test case intended to have the follow-up input (`dateB`) one day after the source input (`dateA`), but it simply hard-codes the two inputs. Without an explicit input transformation program, follow-up inputs cannot be directly generated from automatically generated source inputs. This limitation hinders the reuse of valuable encoded MRs to achieve automated MT and enhance test adequacy. **This paper aims to overcome this obstacle by inferring an explicit input relation from a given test case with its hard-coded input pairs.** Specifically,

our goal is to construct an input transformation function that turns a source input into a follow-up input as shown in Figure 1b. With such input transformations, these encoded MRs can apply to a wider range of test inputs to test SUTs more exhaustively (Figure 1c).

This task can be viewed as a programming by example (PBE) problem, where the aim is to synthesize a transformation function that turns a given source input into the corresponding follow-up input. The challenge lies in *correctly interpreting the contextual information, such as the relationship between hard-coded input pairs, output relations, and the properties of the SUT.* Moreover, with only one pair of source and follow-up inputs available as an example [65], there is a risk of generating program overfitted to the given example instead of realizing the true intention, as noted in existing PBE studies [2, 19, 45]. Therefore, *effectively leveraging contextual information is crucial to guide PBE and generate a generalizable input transformation* that aligns with the *semantic* of the encoded MR, ensuring it applies to all potential source inputs with the corresponding output relation.

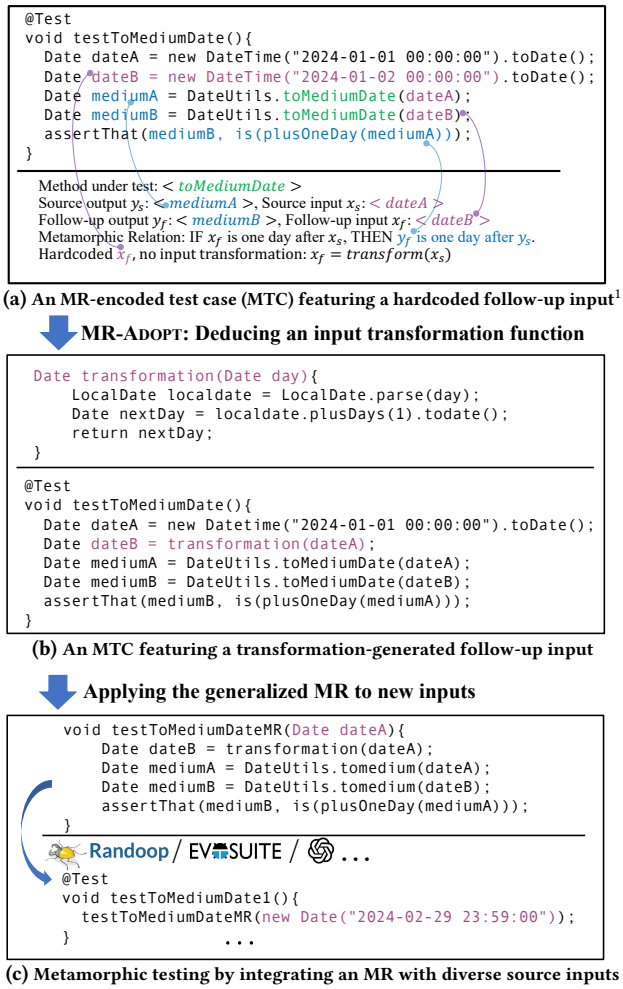
In this paper, we propose MR-Adopt, an approach that leverages large language models (LLMs) to automatically generate input transformation functions for MRs encoded in existing test cases. Trained on extensive code corpora from various domains, LLMs have demonstrated effectiveness in code understanding [18, 35, 39] and generation [7, 14, 23]. Thus, LLMs have the potential to understand contextual information and generate code based on such information. Our insight is to leverage the code understanding ability of LLMs to mine the intention of MR and input relation from the hard-coded test inputs and SUT's function, and take advantage of their code generation ability to produce good input transformation code. We propose three designs to harness LLMs' abilities.

Firstly, we observe that directly providing LLMs with contextual information only results in around 50% generalizable transformations (Section 4.5). This is unsatisfactory. To address this, we need a design that allows LLMs to effectively express the input relation inferred from the hard-coded inputs and generate transformation code. *To realize this goal,* we design MR-Adopt with two phases. In *Phase1*, LLMs perform analogical reasoning [59, 68] on the hard-coded source-followup input pairs to infer new input pairs that obey the same input relation. In *Phase2*, LLMs generate an input transformation function based on (i) the input pair hard-coded by developers and (ii) additional input pairs generated by LLMs in *Phase1*. This design not only enables LLMs to generate code in their familiar setup (where a task description and several examples are provided) [30], but also mitigates the above-mentioned overfitting issue due to the limited number of examples.

Secondly, we found that LLMs often generate task-irrelevant code segments, of which some are even faulty. For example, when tasked with generating a test input, an LLM might include an incorrect assertion statement. MR-Adopt addresses this by refining the LLM-generated code through data-flow analysis, extracting only the relevant code for the given task (i.e., additional input pairs and input transformation generation).

Thirdly, to mitigate the errors in the relevant codes generated by LLMs, MR-Adopt leverages the developer-written output relations

[1]This MR-encoded test case is crafted from `org.hisp.dhis.util` in project DHIS2-core, where long format date is "`yyyy-mm-dd hh:mm:ss`" and medium format date is "`yyyy-mm-dd`".



(a) An MR-encoded test case (MTC) featuring a hardcoded follow-up input[1]

(b) An MTC featuring a transformation-generated follow-up input

(c) Metamorphic testing by integrating an MR with diverse source inputs

Figure 1: Overview of MR-Adopt for Metamorphic Testing

(i.e., assertions) in MTCs as oracles to verify the generated test pairs. We further employ additional inputs to select the most generalizable input transformation as the result.

We evaluated MR-Adopt with 100 developer-written test cases that encode MRs. The results show that MR-Adopt can generate compilable input transformations for 95 MRs, where 72 can generalize to all potential source inputs prepared in our evaluation. MR-Adopt generates 17.28% more compilable transformations and 33.33% more generalizable transformations than directly prompting GPT-3.5. Besides, MR-Adopt-generated transformations produce follow-up inputs for 91.21% source inputs, representing a 122.10% improvement over GPT-3.5 in generating follow-up inputs. Our ablation study indicates that all three designs (i.e., additional input pairs, date-flow analysis based refinement, and output-relation based validation) contribute to MR-Adopt's overall performance, with validation and additional input pairs having the most impact. Furthermore, incorporating MRs with input transformations and new source inputs leads to 10.62% and 18.91% increases in line coverage and mutation score on top of developer-written test cases, demonstrating the practical usefulness of MR-Adopt-generated transformations in enhancing test adequacy.

Contribution. Our work makes the following contribution:

- To the best of our knowledge, we are the first to generate input transformations for MRs encoded in test cases. With the generated input transformations, more encoded MRs can be reused to enhance the test adequacy of SUTs.
- We propose MR-Adopt, an LLM-based approach to deduce input transformation functions. By generating multiple example input pairs, MR-Adopt mitigates overfitting and produces generalizable transformations. It also incorporates a code refinement strategy based on data-flow analysis and a validation strategy to mitigate the faulty irrelevant code generated by LLMs. This design can be applied to other code generation tasks.
- We extensively evaluate MR-Adopt's effectiveness in generating input transformations. Results show that MR-Adopt can generate effective input transformations, where 72% input transformations are generalizable to all prepared source inputs. When integrated with these transformations, the encoded MRs increase line coverage by 10.62% and mutation score by 18.91%.
- We build a dataset of 100 encoded MRs dated after 01-April, 2023, and released it with our replication package on the website [56].

2 PRELIMINARIES

2.1 Metamorphic Testing

Metamorphic Testing (MT) validates a program P using Metamorphic Relations (MRs). An MR $\mathcal{R}$ can be expressed as a logical implication from an ***input relation*** $\mathcal{R}_i$ to an ***output relation*** $\mathcal{R}_o$ [10, 47, 65].

$$\mathcal{R} = \langle \mathcal{R}_i, \mathcal{R}_o \rangle, \text{ where } \mathcal{R}_i\left(x_s, x_f\right) \implies \mathcal{R}_o\left(y_s, y_f\right)$$

$\mathcal{R}_i$ defines the rule to generate an additional test input (known as *follow-up input* x_f) from a given test input (known as *source input* x_s), and $\mathcal{R}_o$ defines the relation between the expected outputs (y_s, y_f) for the source and follow-up inputs, respectively. For example, given a program P implementing the *sine* function, an MR can be defined over an input relation $\mathcal{R}_i$ as $x_f{=}{-}x_s$ ($\forall x_s \in \mathbb{R}$) and an output relation $\mathcal{R}_o$ as $y_f{=}{-}y_s$, based on the property that $P(x){=}{-}P(-x)$ should hold for a correctly implemented *sine* function.

Given an MR $\mathcal{R}$ for a SUT P, conducting MT for P entails the following five steps: (i) constructing a source input x_s, (ii) executing P on x_s and obtaining the source output y_s, (iii) constructing a follow-up input x_f that satisfies $\mathcal{R}_i$, (iv) executing P on x_f and obtaining the follow-up output y_f, and (v) verifying if the two outputs y_s and y_f satisfy the output relation $\mathcal{R}_o$. Typically, a function referred to as the ***input transformation*** is designed to implement $\mathcal{R}_i$ to generate x_f from the given x_s, and x_s can be written by developers or automatically generated (e.g., random testing) [47, 65].

2.2 MR-Encoded Test Cases

MR-encoded test cases (MTCs), introduced by Xu et al. [65], are test cases encoded domain-specific knowledge that suggests useful MRs. These MTCs are prevalent, with over 11,000 identified across 701 open-source projects in their study. An MTC can be considered as an instance of an MR, already implemented with specific source and follow-up input values, invocations of methods under test, and output relation assertions. These output relations are well-coded and serve as oracles, requiring no further refinement to apply to new test inputs. Such encoded MRs can be generalized to new inputs and facilitate automated MT by incorporating automatic input generation techniques.

Consider the example in Figure 1. The encoded MR in this test case is: "IF a date x_1 in long format ("yyyy-mm-dd hh:mm:ss") is one day ahead of another long-format date x_2 ($\mathcal{R}_i$), THEN x_1 in medium format ("yyyy-mm-dd") should also be one day ahead of medium-format x_2 ($\mathcal{R}_o$)". The SUT method `toMediumDate` is executed on the source input `dateA` and the follow-up input `dateB` separately, and the corresponding outputs are verified by `assertThat(mediumB, is(plusOneDay(mediumA))`, which implements $\mathcal{R}_o$.

Such an implemented MR instance can be reused and generalized to many new inputs. However, the follow-up input `dateB` is hardcoded as `"2024-01-02 00:00:00"` instead of being generated from `dateA` by an input transformation program. While the $\mathcal{R}_o$ is explicitly coded, the $\mathcal{R}_i$ remains **implicit**, hidden within the specific source and follow-up input values `dateA` and `dateB`. According to Xu et al.'s study, over 70% of MR-encoded test cases lack explicitly coded $\mathcal{R}_i$ (i.e., input transformations). This limitation prevents these MRs from being directly applied to new inputs automatically generated by existing tools, e.g., Evosuite [17] and Randoop [44]. While these tools are proficient in generating diverse source inputs, they cannot generate input pairs that satisfy an input relation.

In this paper, we address this limitation by deriving an explicit input relation from a given test case and its hardcoded input pairs. Specifically, our goal is to construct an **input transformation function** that converts a source input into a follow-up input, as shown in Figure 1b. With such input transformations, embedded MRs can be reused with a broader range of test inputs (Figure 1c) to exercise more SUT's behaviors, thereby enhancing test adequacy. Additionally, these developer-written MRs serve as reliable oracles for new test generation.

3 MR-ADOPT

Figure 2 presents an overview of MR-Adopt. It takes a pair of source and follow-up inputs, along with its context (i.e., an MR-encoded test case (MTC) and methods under test (MUT)), and outputs an input transformation function. MR-Adopt works in a two-phase pipeline. In the first phase, it generates additional source-follow-up input pairs and uses them as examples to better describe the input relation, which provides useful guidance for the generation of input transformations. In the second phase, it generates input transformation functions based on these example pairs. This setup, familiar to LLMs for code generation tasks, includes a task description and several examples [30], providing more information to effectively guide LLMs in generating generalized transformations.

In each phase, MR-Adopt employs generation, refinement, and validation procedures. In *Phase1*, MR-Adopt first leverages LLMs to generate candidate test input pairs, then refines them based on data-flow analysis to exclude irrelevant code that can contain errors, and finally filters valid input pairs based on output relation assertions. In *Phase2*, MR-Adopt leverages LLMs to generate candidate input transformations based on the input pairs from *Phase1*. These candidate transformations are then refined by removing irrelevant code elements and adding dependencies, and assessed by applying them to additional source inputs. Ultimately, MR-Adopt outputs the most generalizable transformation function.

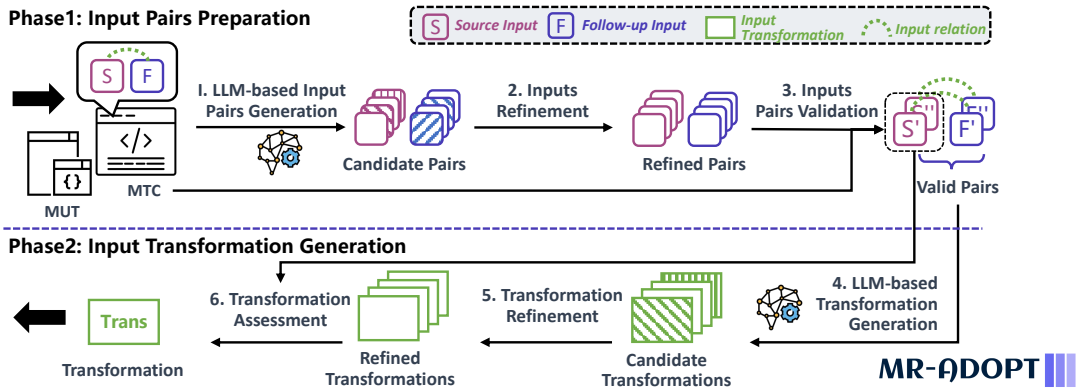


Figure 2: An overview of MR-ADOPT

3.1 *Phase 1:* Input Pair Preparation

3.1.1 Input Pair Generation. MR-ADOPT uses an LLM to produce new source-followup input pairs by imitating a given input pair within the context of an existing MTC (which includes the input pair and developer-written assertions checking the output relation) and corresponding methods under test.

Following the idea of the Chain of Thought strategy [60], MR-ADOPT prompts an LLM in two steps: first to generate source inputs, and then to generate the corresponding follow-up inputs. This step-by-step approach is adopted because we found that LLMs perform better when generating source and follow-up inputs sequentially rather than generating entire input pairs at once. Our source input generation prompt follows recent practices [67, 71], and includes (i) a system message about the role of a Java expert and the task to generate test inputs, (ii) the code of methods under tests (MUTs), (iii) the code of the MR-encoded test case (MTC), and (iv) the output format. Such a prompt provides necessary contextual information (ii and iii) and task description (i and iv) for generating source inputs. Detailed prompt templates and examples are available on MR-ADOPT's website [56]. Listing 1 shows several example source inputs generated by GPT-3.5 with this prompt.

The follow-up input generation prompt is similar to the source input prompt, with the key difference being the addition of previously generated example source inputs to guide the creation of follow-up inputs. We also adjust the task description and output format to instruct LLMs to generate source-followup input pairs, using the original pair in the MTC as a sample. There is a trade-off between providing enough examples to guide the following generation of generalizable transformations and maintaining efficiency in terms of time and cost. To balance these factors, we use five examples. Following recent studies' nucleus sampling [14, 21], MR-ADOPT repeats the above generation process five times with a temperature setting of 0.2 [6, 8]. Listing 2 shows several example input pairs.

3.1.2 Input Pair Refinement. Consistent with observations in existing studies [15], we found that LLM-generated codes often include irrelevant segments, which may introduce errors. For instance, the second input pair in Listing 2 includes an assertion statement `assertEquals(dateTimeB, is(plusOneDay(dateTimeA)));` alongside the desired input pair code. This assertion is irrelevant to constructing `dateA` or `dateB` and incorrectly calls the method

````
## New source input 1:
```java
    Date dateA = new DateTime("2023-12-31 23:59:59").toDate();
```
## New source input 2:
```java
    DateTime dateTimeA = new DateTime("2024-11-30 23:59:59");
    Date dateA = dateTimeA.toDate();
```
## New source input 3:
```java
    Date dateA = new DateTime("2024-01-01 00:00:00").toDate();
```
        ...(other inputs are ommitted)...
````

Listing 1: Examples of LLM-generated source inputs

````
The inferred relation between the source input `dateA` and the follow-up input `dateB` is that `dateA` is one day ahead of `dateB`.
Let's construct the follow-up inputs for the new source inputs:
## New input pair 1:
```java
    Date dateA = new DateTime("2024-12-31 23:59:59").toDate();
    Date dateB = new DateTime("2025-01-01 23:59:59").toDate();
```
## New input pair 2:
```java
    DateTime dateTimeA = new DateTime("2024-11-30 23:59:59");
    Date dateA = dateTimeA.toDate();
    DateTime dateTimeB = new DateTime("2024-12-01 23:59:59");
    Date dateB = dateTimeB.toDate();
    assertEquals( dateTimeB, is(plusOneDay(dateTimeA)) );
```
## New input pair 3:
```java
    Date dateA = new DateTime("2024-01-01 00:00:00").toDate();
    Date dateB = new DateTime("2025-01-01 00:00:00").toDate();
```
        ...(other input pairs are ommitted)...
````

Listing 2: Examples of LLM-generated input pairs

`plusOneDay(Date date)` with a `DateTime` object, resulting in a type mismatch exception.

Our task focuses on constructing source and follow-up inputs. To exclude such irrelevant code and bypass unnecessary errors, we perform a data-flow analysis on the LLM-generated code and build a dependency graph. MR-ADOPT constructs data dependency graphs for source/follow-up inputs by first identifying statements that define source/follow-up inputs, extracting variables used for these definitions, and then recursively tracing these variables' dependencies. Finally, MR-ADOPT isolates the dependent statements related to the source and follow-up inputs and removes other statements. For example, in the second input pair of Listing 2, the source input `dateA` and follow-up input `dateB` depend on objects `dateTimeA`

```
@Test
void testToMediumDate(){
    // LLM-generated new source input
    Date dateA = new DateTime("2024-12-31 23:59:59").toDate();
    // LLM-generated corresponding follow-up input
    Date dateB = new DateTime("2025-01-01 23:59:59").toDate();
    Date mediumA = DateUtils.toMediumDate(dateA);
    Date mediumB = DateUtils.toMediumDate(dateB);
    assertThat(mediumB, is(plusOneDay(mediumA)));
}
```

Listing 3: Validating an LLM-generated input pair

and `dateTimeB`, respectively. Thus, the statements (Lines 11-14) for constructing `dateA`, `dateTimeA`, `dateB`, and `dateTimeB` are considered relevant, while the assertion statement (Line 15) is excluded. Finally, MR-Adopt retains only the statements relevant to constructing source and follow-up inputs, excluding all other irrelevant statements from the LLM-generated code.

3.1.3 Input Pair Validation. The previous refinement step removes the irrelevant code segments generated by LLMs and results in candidate source-followup input pairs. However, a pair of inputs still can be *invalid* if they violate the input relation of an encoded MR. For example, the third input pair shown in Listing 2 is an invalid test pair. The input relation of the embedded MR is that "`dateA` is one day ahead of `dateB`", while an LLM generates an input pair of "2024-01-01" and "2025-01-01". Such test case pairs do not align with the intended input transformation and can mislead the generation of transformations. To address this, we propose a method to discard such invalid input pairs.

We use the output relation of an encoded MR to validate LLM-generated input pairs. Specifically, MR-Adopt executes SUT on generated input pairs and checks the outputs against the output relation, which is an explicit reusable code in the MTC, i.e., the developer-written assertions (Line 7 in Figure 1a). For each input pair, if its outputs of invoking methods under test on the inputs pass the developer-written assertions, MR-Adopt considers it a valid input pair. As shown in Listing 3, if the outputs `mediumA` and `mediumB` pass the assertion (Line 9), the inputs `dateA` and `dateB` are considered valid. This step aims to filter out invalid input pairs generated by LLMs from the example set. It could discard some source-followup input pairs that match the input relation in fact. Factors such as the bugs in a non-regression SUT may lead to false violations and mistaken deletions of these pairs. However, the goal of the first phase is to prepare examples that give more information about the input relation for the second phase. Thus, it does not require *complete* source-followup input pairs.

3.2 *Phase 2:* Transformation Generation

3.2.1 Transformation Generation. In this step, MR-Adopt instructs an LLM to generate candidate input transformation functions for an encoded MR by providing example source-followup input pairs. The examples include the original hard-coded pair and additional pairs generated in *Phase1*. The prompt for transformation generation is similar to the input pair generation prompt (Section 3.1.1), consisting of (i) a system message, (ii) the code of MUT, (iii) example input pairs, (iv) the code of an MTC, and (v) the output format. The difference is that the task shifts from generating source-followup input pairs to generating input transformation functions, whose parameter list and return type are already specified. Detailed prompt template and samples are available on MR-Adopt's website [56].

````
# OUTPUT FORMAT
Generate the transformation function by complementing the following
code skeleton.

    ```java
    public static Date transformation(Date day)  {
        // TODO
        Date nextDay =
        return nextDay;
    }
    ```
````

Listing 4: An example of output format in the prompt

Listing 4 shows the output format specified in the prompt, which defines the skeleton of the input transformation function to generate. It includes the function name, parameter (i.e., source input) types and names, and type of the return value (i.e., follow-up input) [2]. Following recent studies' nucleus sampling [14, 21], for each MR, MR-Adopt instructs an LLM to generate one input transformation function, and repeats the generation process five times with a temperature setting of 0.2 [6, 8]. Finally, five candidate transformation functions can be generated.

MR-Adopt extracts the generated functions by identifying code blocks wrapped with ``` and extracting the code that matches the given transformation function skeleton. This ensures the generated code conforms to the required format and can be easily integrated into given MR-encoded test cases.

3.2.2 Transformation Refinement. Similar to the situation discussed in Section 3.1.2, LLM-generated transformation functions can contain irrelevant code, some of which can cause errors (e.g., invoking non-existing APIs). To address this issue, MR-Adopt constructs data dependency graphs for follow-up inputs by first identifying statements that define these inputs. It then extracts the variables used in these definitions and recursively traces their dependencies. Statements involved in the dependency graph are considered relevant and retained, while others are considered irrelevant and excluded. As the example shown in Listing 5, the follow-up input `nextDay` depends on `localDate`, which further depends on `day`. Statements constructing `nextDay` and `localDate` are retained, while irrelevant statements such as `Date dayAfter=day.after(1)` are excluded.

````
The transformation function can be implemented as follows:

```java
public static Date transformation(Date day)  {
    Date dayAfter = day.after(1); // non-exisitng API and irrelevant
    LocalDate localdate = LocalDate.parse(day);
    Date nextDay = localdate.plusDays(1).todate();
    return nextDay;
}
```
````

Listing 5: An example of LLM-generated transformation

After excluding irrelevant code, MR-Adopt analyzes and imports dependencies needed by the generated transformation function. Using JavaParser [24], MR-Adopt identifies dependent class names through syntax analysis. It then retrieves potential classes defined or imported in source and imports those whose names match the dependent classes. For example, the internal class `LocalDate` Listing 5 will be imported. Although matching dependent classes by name can be inaccurate if LLM-generated code uses incorrect names or

[2] For MRs with multiple follow-up inputs, the return type is a list of objects.

```
@Test
void testToMediumDate() {
    // new source input
    Date dateA = new Date("2024-02-01 00:00:00");
    // invoking generated transformation on new source input
    Date dateB = transformation(dateA);
    Date mediumA = DateUtils.toMediumDate(dateA);
    Date mediumB = DateUtils.toMediumDate(dateB);
    assertThat(mediumB, is(plusOneDay(mediumA)));
}
```

Listing 6: Validating an LLM-generated input transformation with a new source input

```
public static Date transformation(Date day) {
    int dayValue = day.getDate();
    int monthValue = 1; // set the month to 1 since we don't know
    ↪ which month the input represents
    int yearValue = day.getYear();
    Date nextDay = new Date(year, month, day +1);
    return nextDay;
}
```

Listing 7: An example of LLM-generated transformation

third-party libraries not imported in the project, our experimental observations indicate that this issue is minor. Most dependent classes are derived from the context provided to LLMs, such as the MUT or MTC code.

3.2.3 Transformation Assessment. After refining candidate transformations, MR-Adopt further assesses their quality by applying them to new source inputs. In this step, MR-Adopt leverages new source inputs generated in Section 3.1 to assess the generalizability of candidates and then selects the most generalizable one.

Specifically, MR-Adopt uses new source inputs as test inputs and employs developer-written assertions (i.e., output relation assertions) as test oracles. A transformation is considered applicable to a given source input if (a) the input transformation function can successfully generate a corresponding follow-up input without throwing exceptions, and (b) the outputs from executing the methods under test pass the developer-written assertions. For example, in Listing 6, given the source input `dateA`, if the follow-up input `dateB` is successfully generated and the outputs `mediumA` and `mediumB` pass the assertion (Line 9), MR-Adopt considers the transformation applicable to input `dateA`. Conversely, Listing 7 shows a failing transformation that only works for January dates. MR-Adopt assesses all candidate transformation functions using both new and the original source input. It then selects the most generalizable transformation that applies to the most inputs. In case of a tie, MR-Adopt returns the first generated one as the result.

4 EVALUATION

4.1 Research Questions

Our evaluation aims to answer the following research questions:

- **RQ1:** *How effective is MR-Adopt in generating input transformations?* This RQ compares the quality of the input transformation functions generated by MR-Adopt and baselines to evaluate the effectiveness of MR-Adopt in generating generalizable input transformations for MRs encoded in test cases.
- **RQ2:** *How effective are MR-Adopt-generated input transformations in constructing follow-up inputs, compared with LLMs?* This RQ investigates the benefits of generating input transformation functions, by comparing the quality of follow-up inputs produced by these functions versus those directly generated by LLMs.
- **RQ3:** *What is the contribution of each component in MR-Adopt?* This RQ performs an ablation study to reveal how each component contributes to generating input transformations.
- **RQ4:** *How useful are encoded MRs in enhancing test adequacy with the generated input transformations?* With input transformations generated by MR-Adopt, more encoded MRs can be reused with new inputs to test more behaviors of SUT. This RQ investigates the usefulness of such encoded MRs in improving test adequacy, demonstrating the usefulness of generating input transformation.

4.2 Dataset

MR-encoded test cases (MTCs). We followed Xu et al. [65] to collect high-quality Java projects with at least 200 stars over GitHub. Besides, we further excluded the projects created before 01-April 2023 to prevent the experimental LLMs from having potentially learned the code during training, thereby reducing the potential for data leakage [6]. Finally, we collected 2,007 MTCs from qualified projects. From these MTCs, we retained test cases that (i) can be successfully compiled, (ii) can be successfully executed (i.e., passing developer-written assertions), and (iii) contain MRs associated with exactly two method invocations (one for the source input and one for the follow-up input). The third criterion excludes complex and less common MRs involving multiple input groups [65]. MRs with exactly two invocations constitute the majority (65%) of MTCs [65]. Handling MRs with more than two invocations presents significant challenges in identifying source and follow-up inputs. This problem is nontrivial and remains an important future work. Finally, we obtained 180 MTCs, including 54 with explicit input transformations written by developers and 126 without such transformations, consistent with the distribution reported by Xu et al. [65].

Generation Tasks and Ground Truths. Based on the collected 180 MTCs, we prepared a dataset containing (i) 100 MTCs without input transformations as tasks, and (ii) corresponding input transformation functions as ground truths. The preparation process is as follows. Firstly, we tried to utilize all 54 MTCs with ground truths, i.e., developer-written input transformations. For each MTC, we executed the input transformation on the hardcoded source input to obtain the follow-up input. We prepared a task by replacing the developer-written transformation with the hardcoded follow-up input. Some MTCs whose follow-up input cannot be hardcoded are excluded. For example, an MR for a text render class is "the width of a text (source input) should not be greater than its bold version (follow-up input)". The follow-up input (bold text) can only be generated by a method `bold()`, which is a developer-used transformation program. Finally, we built 36 tasks from 36 MTCs with developer-written transformations.

Next, we manually constructed input transformation functions for MTCs lacking developer-written transformations. Specifically, 64 out of 126 MTCs without input transformations were randomly selected as tasks. For each task, one author of this paper examined the SUT and its underlying MRs and then created a transformation function applicable to the original source input and generalizable to new valid source inputs. Another author reviewed these transformations, and any disagreements were discussed and resolved with consensus. This process took approximately 200 human hours. Details of this dataset can be found on MR-Adopt's website [56].

Table 1: Effectiveness of MR-Adopt in generating input transformations for 100 MRs encoded in test cases

Metric (# Trans.)	Direct Prompting			MR-Adopt		
	Llama3	*Deepseek*	*GPT-3.5*	*Llama3*	*Deepseek*	*GPT-3.5*
compilable	79	80	81	86 (+8.86%)	89 (+11.25%)	**95 (+17.28%)**
>0% generalizable	69	72	69	77 (+11.59%)	82 (+13.89%)	**83 (+20.29%)**
>75% generalizable	64	67	63	74 (+15.66%)	80 (+19.40%)	**81 (+28.57%)**
100% generalizable	57	60	54	68 (+19.30%)	71 (+18.33%)	**72 (+33.33%)**

n% generalizable: the number of generated input transformations applicable to at least *n*% of source inputs.

4.3 Environment and Large Language Models

Our experiments were conducted on machines with three RTX4090 GPUs, dual Intel Xeon E5-2683 v4 CPUs, and 256 GB RAM.

The large language models used in our evaluation include GPT-3.5 from OpenAI [42] and three open-source code models: Llama3-8B [37] from Meta, Deepseek-coder-7b [11] from DeepSeek, and CodeQwen1.5-7B-Chat [1] from Alibaba. We use these LLMs since they are popular state-of-the-art [16] code models in well-known LLM families and deployable at our machines.

4.4 Source Input Preparation

To evaluate the generated input transformations, we need new valid source inputs as a "test set". As introduced in Section 2.1, automatic test input generation techniques (such as Evosuite [17] or Randoop [44]) can be employed to prepare source inputs. However, we found that these tools often fail to generate test inputs for many MRs. This is because over 50% of experimental MRs' inputs are user-defined complex objects with complicated preconditions and environments, which are challenging for tools like Evosuite to handle. This aligns with Xu et al.'s observation [65].

Recent studies show that LLMs are good test input generators [67, 71]. In this study, we employed an LLM (Qwen) to generate new source inputs for evaluating transformations, while other experimental LLMs were used to generate transformations. As a reminder, to mitigate circular evaluation, we employed different LLMs for preparing the "test set" and for generating input pairs and transformation functions in MR-Adopt. We reused the prompt template from MR-Adopt's *Phase1*. Qwen was instructed to generate five source inputs at a time, and the process was repeated ten times with a 0.2 temperature setting to produce more source inputs.

For the 100 experimental MRs, Qwen generated a total of 5,355 new source inputs. We first filtered out 3,058 duplicate inputs using string matching. Next, we identified valid source inputs by executing them on the corresponding ground truth transformations. A source input is considered valid only if the ground truth transformation successfully generates a follow-up input, and the outputs of this source input and corresponding follow-up input pass the developer-written assertions (R_o). Qwen failed to generate a new valid source input for 5 MRs whose inputs are complex objects and have strict domain-specific constraints. Finally, we collected 1,366 valid source inputs, averaging 14.37 per MR.

4.5 RQ1: Effectiveness of MR-Adopt

4.5.1 Experiment Setup. This RQ inspects MR-Adopt's effectiveness in generating input transformation functions by examining their comparability and generalizability to new source inputs.

Baselines. To the best of our knowledge, no existing approach generates input transformation functions for MRs across different domains. Given the proven effectiveness of LLMs in code and test generation, we set *directly prompting LLMs* as a baseline. Specifically, we directly prompted GPT-3.5-turbo-0125, Llama3-8B-Instruct, and Deepseek-coder-7b-instruct-v1.5 (shorten as GPT-3.5, Llama3, and Deepseek, respectively). The template is similar to MR-Adopt's and available at [56]. The knowledge cut-off dates for these models are September 2021 [43], March 2023 [38], and March 2023 [12], respectively, before the creation date of our dataset's MTCs (Section 4.2),

Configuration of Baseline LLMs. Following recent studies [14], we used the nucleus sampling [21] and repeated the generation process five times for each task with a temperature setting of 0.2 [6, 8], and selected the best result for comparison. The configuration of MR-Adopt was introduced in Section 3.2.1.

Metrics. For this RQ, we introduced two metrics: (i) *# compilable transformations:* the number of generated input transformations that can successfully compile, and (ii) *# n% generalizable transformations:* the number of generated input transformations applicable to at least *n*% of source inputs prepared in Section 4.4 ($n = 0, 75, 100$ representing at least one, upper-quartile, and all inputs, respectively). A transformation t is considered *applicable* to a source input x_s if t generates a follow-up input x_f for x_s, so that a *correct* SUT does not violate the output relation on the input pair $<x_s, x_f>$.

4.5.2 Result. As shown in Table 1, MR-Adopt effectively produced many compilable input transformation functions that well generalize to prepared source inputs. We found that MR-Adopt works best with GPT-3.5. Specifically, using GPT-3.5 (the last column), MR-Adopt produced compilable transformations for 95 out of 100 MRs, with 72 of these transformations effectively applied to *all* prepared source inputs. MR-Adopt also works well with Llama3 and Deepseek, generating 68 and 71 (100%) generalizable transformations, respectively. Besides, we found that some generated transformations generalize well to some, but not all, source inputs prepared in our experiment. Specifically, with GPT-3.5, 83 out of 95 compilable transformations applied to at least one source input, and 81 of them applied to more than 75% of the prepared source inputs. Similar results were found with Llama3 and Deepseek. We considered these transformations generated by MR-Adopt *still useful* to some extent, as they successfully prepare some valid input pairs. Upon further analysis, we found that their limitations could potentially be addressed with more comprehensive prompts to handle corner cases. LLM-generated transformations effectively handle common cases but struggle with edge cases. For example, an ideal transformation would generate a higher version string in any scenario (e.g., transforming `"1.0-A1"` to `"1.0-B1"`), but the LLM-generated transformation relies on a 'Major.Minor.Revision' convention (e.g., `"1.0.1"`) and fails with cases like `"1.0-A1"`.

There were 5, 14, and 11 transformations generated by MR-Adopt with GPT-3.5, Llama3, and Deepseek, respectively, that failed to compile. The main reasons include: (i) the generated transformations invoke non-existing methods to generate the follow-up input and (ii) they invoke inaccessible APIs due to permission restrictions (e.g., private methods). Additionally, the compilable but not generalizable transformations were primarily due to *Phase1* failing to

Table 2: Effectiveness of MR-Adopt's transformations in constructing follow-up inputs for 1366 source inputs

MR-Adopt	Llama3	Deepseek	GPT-3.5	Improvement
1246	697	724	597	+72.10%∼**+108.71%**
MR-Adopt	**Llama3$^+$**	**Deepseek$^+$**	**GPT-3.5$^+$**	**Improvement**
1246	770	737	708	+61.82%∼**+75.99%**

$^+$ means incorporating MR-Adopt's input refinement procedure for LLMs' answers.

generate valid input pairs for these MRs, leading to LLM-generated transformations overfitted to the given input pair.

We also compared MR-Adopt's performance (columns 5-7) with the baseline of directly prompting LLMs (columns 2-4). Although the output relation encoded in MTC was provided in prompts for baselines to aid transformation generation, MR-Adopt still generated more compilable transformations. This improvement is due to MR-Adopt's code refinement and assessment strategies. Moreover, MR-Adopt demonstrates substantial improvements in generating transformations that are >75% and 100% generalizable, with increases of 15.66% to 28.57% and 18.33% to 33.33%, respectively. This suggests the effectiveness of preparing more examples for LLMs and the benefits of MR-Adopt's refinement and selection strategies.

Answer to RQ1: MR-Adopt significantly outperforms the baseline LLMs across all metrics. Compared to directly prompting LLMs, MR-Adopt achieves 18.33%∼33.33% improvement in generating 100% generalizable input transformations.

4.6 RQ2: Effectiveness of Input Transformations

4.6.1 Experiment Setup. This RQ examined the quality of follow-up inputs produced by input transformations generated by MR-Adopt. We set LLMs as the baselines because they are off-the-shelf black-box transformations that can generate follow-up inputs given source inputs, as introduced in Section 3.1.1. We also included LLMs enhanced with MR-Adopt's refinement procedure (marked with $^+$) for comparison. This can reflect the effectiveness of MR-Adopt's refinement for input pairs preparation (Section 3.1.2).

Metric. We generated follow-up inputs by feeding the 1,366 prepared source inputs (Section 4.4) to input transformations generated by MR-Adopt and the vanilla LLM baselines. To compare the qualities of the follow-up inputs produced by the MR-Adopt-generated transformations and the baselines, we used the number of *valid* follow-up inputs as the metric. Similar to Section 4.5, we consider a follow-up input x_f *valid* if it and its corresponding source input can pass developer-written output relation assertions.

4.6.2 Result. As shown in Table 2, when built with GPT-3.5, input transformation functions generated by MR-Adopt produced valid follow-up inputs for 1246 out of 1366 (91.22%) source inputs. The high validity rate demonstrated that MR-Adopt contributed to abundant useful source-followup input pairs.

In comparison, three vanilla LLMs only generated valid follow-up inputs for 697 (51.02%), 724 (53.00%), and 597 (43.70%) source inputs, respectively. MR-Adopt surpassed them by 72.10%-108.71%. LLMs enhanced with MR-Adopt's input refinement procedure (marked with $^+$) worked better than the vanilla LLMs. This indicates the usefulness of our design to refine the LLM-generated test inputs (Section 3.1.2). Meanwhile, MR-Adopt's transformations still outperformed the enhanced LLMs by generating 61.82% more valid follow-up inputs than Llama3$^+$, 69.06% more than Deepseek$^+$, and 75.99% more than GPT-3.5$^+$. This significant performance gap highlights the effectiveness of MR-Adopt's transformation functions compared to the state-of-the-art LLMs. It also evidenced the usefulness of our idea to codify the input transformation by leveraging the code understanding and generation abilities through the two-phase pipeline and preparation-refinement-validation process.

We also summarized two major limitations of using vanilla LLMs as black-box transformations based on our observation. Firstly, LLMs can generate a follow-up input with a wrong value, which is similar to the case in Listing 2. Another limitation is that LLMs often fail to capture the constraints between multiple arguments of the follow-up input. For instance, consider a method `deserial(data, size)` to deserialize an `ArrayList data` with a given `size`. The `size` should not be greater than the length of `data`. However, LLMs may miss this constraint and generate invalid value for `size`. These issues about value processing could be due to LLMs' limited inference ability. Instead, MR-Adopt asks LLMs to codify the input transformation and uses the code to do calculation and processing, which is recognized as a better way to exert LLMs' abilities [28]. Besides, we argued that using LLMs as transformations can be costly since we need to request LLMs for each source input. Meanwhile, MR-Adopt uses LLMs to generate transformations for once, and there is no need to query LLMs when using the generated transformations.

Answer to RQ2: MR-Adopt's refinement step can effectively enhance follow-up input generation, with up to 18.59% improvement for GPT-3.5. Additionally, MR-Adopt-generated transformations can effectively generate follow-up inputs for 91.21% source inputs, surpassing GPT-3.5$^+$ by 75.99%.

4.7 RQ3: Ablation Study on MR-Adopt

4.7.1 Experiment Setup. We created three variants v_1, v_2, and v_3 of MR-Adopt by ablating three components to analyze the helpfulness of these designs for generating generalizable input transformations. We chose MR-Adopt built with GPT-3.5 which achieves the best result in RQ1 (Section 4.5). The variants are as follows:

- v_1**: MR-Adopt w/o additional input pairs.** This variant used only one source-followup input pair hard-coded in an MTC to guide the input transformation generation. It did not use additional input pairs generated in MR-Adopt's *Phase1* (Section 3.1).
- v_2**: MR-Adopt w/o refinement step.** This variant disabled the refinement step for generated input transformations in MR-Adopt (Section 3.2.2).
- v_3**: MR-Adopt w/o assessment step.** This variant disabled the assessment step for selecting the most generalizable transformations (Section 3.2.3). Instead, it randomly selected one of the compilable transformation functions as the result.

4.7.2 Result. As shown in Table 3, removing additional input pairs (v_1) led to a 19.44% decrease in generating 100% generalizable transformations. This suggests that additional input pairs effectively mitigate the overfitting problem caused by the limited examples

Table 3: Contribution of each component in MR-Adopt

Metrics (# Trans.)	MR-Adopt	v_1: w/o input pairs	v_2: w/o refinement	v_3: w/o assessment
compilable	95	87 (-8.42%)	93 (-2.10%)	95 (0.00%)
>0% generalizable	83	73 (-12.04%)	82 (-1.20%)	70 (-15.66%)
>75% generalizable	81	66 (-18.51%)	75 (-7.40%)	59 **(-27.16%)**
100% generalizable	72	58 **(-19.44%)**	61 **(-15.27%)**	56 (-22.22%)

in PBE [2, 19, 45], helping MR-Adopt generate more generalizable transformation.

Similarly, disabling the refinement step (v_2) reduced 15.27% input transformations that generalize to 100% prepared inputs. This indicates that some generated transformations have minor issues and can be refined by excluding irrelevant code. Besides, disabling the assessment step (v_3) decreased 22.22% input transformation generalizable to 100% inputs. This suggests that, even with additional input pairs and refinement, few 100% generalizable transformations can be generated, and random selection may miss them. The assessment step is necessary to rank the most generalizable function.

Answer to RQ3: All three designs contribute to the effectiveness of MR-Adopt in generating generalizable transformations. The assessment procedure contributes the most, and additional example input pairs contribute similarly.

4.8 RQ4: Usefulness of Input Transformations

4.8.1 Experiment Setup. In this RQ, we integrated the generated input transformations into MTCs to construct generalized MRs and measured how well such MRs enhanced test adequacy. This demonstrated the practical usefulness of MR-Adopt's transformations in enhancing test adequacy.

New Test Cases Construction. We applied generalized MRs to the automatically generated source inputs introduced in Section 4.4 to obtain a set of new test cases (denoted as $\mathcal{M}$). We compare such test cases against two baselines: (i) the developer-written test cases (i.e., MTCs) (denoted as $\mathcal{D}$) and (ii) test cases based on the LLM-generated source and follow-up input pairs (denoted as $\mathcal{L}$). Specifically, we combined the prepared source inputs (Section 4.4) with valid follow-up inputs generated by Llama3$^+$ which performed the best in RQ2 (Section 4.6). Considering generalized MR based test cases and LLM-generated input pairs based test cases are extended from developer-written existing test cases, we followed Xu et al. [65]'s practice to analyze the test adequacy improvement on top of developer-written test cases.

Metrics. We measured test adequacy using two metrics: (i) Line Coverage – percentage of code lines in target classes executed, and (ii) Mutation Score – percentage of mutants killed by test cases.

Mutation Testing: We employed Pitest [46] to conduct mutation testing. Each MR only focused on one or two methods under test in the target class. To include the covered lines or killed mutants in the methods intransitively invoked by MR-involved methods for comparison, we employed Pitest to generate mutants targeting all methods in a target class. Finally, Pitest successfully generated 4,388 mutants for 45 target classes covered by 88 MRs in the dataset (Section 4.2). Pitest failed for the other 12 MRs' classes because of environmental issues (e.g., conflict dependencies).

Table 4: Enhancement of test adequacy from generalized MR based test cases ($\mathcal{M}$) on top of developer-written ($\mathcal{D}$) and LLM-generated input pairs ($\mathcal{L}$) based test cases

Metrics	VS. $\mathcal{D}$			VS. $\mathcal{D}$+$\mathcal{L}$		
	$\mathcal{D}$	$\mathcal{D}$+$\mathcal{M}$	Improve.	$\mathcal{D}$+$\mathcal{L}$	$\mathcal{D}$+$\mathcal{L}$+$\mathcal{M}$	Improve.
Line Coverage	0.2373	0.2625	+10.62%	0.2588	0.2698	+4.25%
Mutation Score	0.1322	0.1572	+18.91%	0.1710	0.1807	+5.67%

4.8.2 Result. As shown in Table 4, compared to developer-written MTCs ($\mathcal{D}$), incorporating new test cases constructed from generalized MR ($\mathcal{D}$+$\mathcal{M}$) increased the line coverage by 10.62% and the mutation score by 18.91%. This suggested that MR-Adopt could enhance the test adequacy by integrating high-quality test oracles (i.e., output relation of the encoded MR) with a diverse set of potential test input pairs of the MR ($\mathcal{M}$). Although the developer-written test inputs hard-coded in MTCs were carefully crafted and invaluable, each typically included one pair of test inputs and could not sufficiently exercise the SUT's behaviors. The new source inputs generated by test generation techniques and the corresponding follow-up inputs enabled by MR-Adopt may reach program states not covered by the hard-coded inputs.

Besides, by analyzing the benefit of using MR-Adopt ($\mathcal{D}$+$\mathcal{L}$+$\mathcal{M}$) over the test suite enhanced by LLM-generated valid input pairs ($\mathcal{D}$+$\mathcal{L}$), we could still observe 4.25% and 5.67% improvements in the line coverage and the mutation score, respectively. This suggested that even if an LLM could act as a black-box transformation to generate some valid source-followup inputs and reach more execution states of SUT, MR-Adopt could generate input transformations that apply to more source inputs and better enhance the test adequacy.

Answer to RQ4: Test cases constructed from generalized MRs could achieve 10.62% and 18.91% increases in the line coverage and mutation score, respectively, demonstrating generalized MRs' practical usefulness in enhancing test adequacy.

5 DISCUSSION

5.1 Threads to Validity

We identified potential threats to the validity of our experiments and have taken measures to mitigate them.

Representativeness of Experimental Subjects. A potential threat is whether our evaluation findings can generalize to different projects. To mitigate this threat, we adopted the criteria from existing studies [22, 58, 65] to select high-quality and well-maintained Java projects as representative subjects (Section 4.2) and evaluated our method on these projects. Besides, evaluating LLMs with subjects seen during model training (known as the data leakage issue) will make the findings biased [61]. To mitigate this threat, we collected MR-encoded test cases created after the training cut-off date of the experimental LLMs, as described in Section 4.2.

Representativeness of Experimental LLMs. Our method depends on LLMs, and we also use LLMs as baselines. A potential threat is whether our evaluation findings based on the selected LLMs are representative. To mitigate this threat, we evaluated our method with LLMs from three well-known LLM families, i.e., GPT-3.5 from OpenAI, Llama3 from Meta, and Deepseek from DeepSeek. They represent the state-of-the-art code LLMs (according to the

EvalPlus leaderborad) that can be deployed with the hardware capacity of our machine, as introduced in Section 4.3.

Quality of the Experimental Source Inputs. As introduced in Section 4.4, we used an LLM to prepare new source inputs to assess the generalizability of generated input transformations. Low-quality source inputs may threaten the evaluation validity. To mitigate this issue, we employed another SOTA code LLM (i.e., Qwen) which is not the experimental subject to prepare the source inputs. We then use the ground truth input transformations to filter out invalid source inputs.

Quality of Ground Truths. Besides directly using developer-written input transformations in MTCs (if available) as ground truths, we also manually prepared ground truths for MTCs without input transformations. There is a potential threat regarding the quality of our prepared ground truths. To mitigate this threat, two authors (PhD students) proficient at MT and with more than four years of Java programming experience implemented the ground truths after understanding the intention of the SUTs and the encoded MRs. Specifically, a ground truth was developed by one participant and reviewed by the other until a consensus was reached. Furthermore, the developed ground truths are validated against the original source input.

5.2 Distinct Advantages of MR-based Tests in Fault Detection

Detecting faults in "non-testable" programs. MR-based tests offer distinct advantages in validating *non-testable programs* whose expected outputs for given inputs are hard to specify [10, 47]. The usefulness of MR-based tests in detecting such faults for such programs has been reported in studies [9, 32, 34, 48]. MR-Adopt targets encoded MRs for testing Java classes. We provide two examples to illustrate this advantage.

As shown in Listing 8, the class `AES` contains methods to encrypt and decrypt a string. The `encrypt` function includes a fault: it mistakenly encrypts the `secret` argument instead of the intended `source`. However, the expected string literal after encryption is difficult to specify. This makes it difficult to construct an explicit test oracle based on the expected output to effectively validate the behavior of `encrypt`.

We collected developer-written tests, EvoSuite-generated tests (following Xu et al.'s practice [65]), and LLM-generated tests (following Yuan et al.'s practice and generating tests with GPT-4 [70]). As shown in Listing 9, we found that the non-MR test (including developer-written and LLM-generated assertions) only checks that the encrypted string is not null or empty. EvoSuite failed to generate any assertions. However, this test is weak in validating whether the encryption process is correctly implemented. It ensures only the existence of a non-empty output.

In contrast, the developer-written MR-based test validates the encrypted string using an MR: $x = AES.decrypt(AES.encrypt(x))$ — *IF* an input x is encrypted and subsequently decrypted, *THEN* the final result should be x. The MR-based test successfully detects the fault in encrypting the `secret` argument instead of the `source`, while the non-MR tests fail to detect the fault.

[3]Available in the CipherHelper.java file from the project FlowCI/flow-core-x

```java
public static abstract class AES {
    public static String encrypt(String source, String secret) {
        ...
        cipher.init(Cipher.ENCRYPT_MODE, key);
        byte[] bytes = cipher.doFinal(toBytes(secret)); // BUG: This
        ↪  should be "toBytes(source)" instead of "toBytes(secret)"
        return Base64.getEncoder().encodeToString(bytes);
    }
    public static String decrypt(String encrypted, String secret) {
    ...
}
```

Listing 8: Faulty class `AES`[3] for encrypting and decrypting

```java
public void AES_NonMRTest() {
  String source = "!@#!@#!@1fsd"; String secret = "ssdkF$HUy2A#D%kd";
  String encrypted = CipherHelper.AES.encrypt(source, secret);
  // Developer-written non-MR assertion
  assertFalse(Strings.isNullOrEmpty(encrypted));

  // LLM-generated non-MR assertion
  assertNotNull(encrypted); assertFalse(encrypted.isEmpty());

  // EvoSuite-generated non-MR test: no assertion
  String string0 = CipherHelper.AES.encrypt("jlyN5n~l(%", "AES");
  String string1 = CipherHelper.AES.decrypt("n<h!/N*WHF", ")");
}

//Developer-written MR-based test, MR: x=AES.decrypt(AES.encrypt(x))
public void AES_MRTest() {
  String source = "!@#!@#!@1fsd"; String secret = "ssdkF$HUy2A#D%kd";
  String encrypted = CipherHelper.AES.encrypt(source, secret);
  String decrypted = CipherHelper.AES.decrypt(encrypted, secret);
  assertEquals(source, decrypted);
}
```

Listing 9: Non-MR and MR based tests[4] for class `AES`

Similarly, when testing the `getRegistryCenterTime`[5] function, which is designed to (i) create a new registry entry and return the creation time, or (ii) update an existing entry and return the updating time, it is difficult to determine the expected output because the exact system time depends on when the code is executed.

The non-MR test, including developer-written and LLM-generated assertions, weakly validates if the registration time is greater than 0 and less than the current system time. EvoSuite failed to generate any assertions. Such a weak non-MR test has a significant limitation: it fails to validate whether the function correctly handles the update of existing entries. In other words, it does not explicitly check the chronological order of the successive calls.

In contrast, a developer-written MR-based test validates the registration time based on an MR: *IF* $t_2 = getRegistryCenterTime(key)$ is called after $t_1 = getRegistryCenterTime(key)$, *THEN* $t_1 < t_2$. When these tests are applied to a faulty `getRegistryCenterTime` implementation that returns an unmodified time due to a missing entry update, only the MR-based test can detect this fault, while non-MR tests cannot.

Exercising a wide range of inputs even with a single MR. Another advantage of MT is that *one MR can be applied to a wide range of automatically generated test inputs (known as source inputs) to exercise various program behaviors.*

Our experimental results show that useful tests can be constructed from MRs by leveraging MR-Adopt's input transformations and the wide range of generated test inputs. The constructed

[4]Available in the CipherHelperTest.java file from the project FlowCI/flow-core-x

[5]Available in the `ZookeeperRegistryCenter` class within the project apache/shardingsphere-elasticjob

[6]Available in the ZookeeperRegistryCenterQueryWithoutCacheTest.java file from the project apache/shardingsphere-elasticjob

```java
public void GetRegistryCenterTime_nonMRTest() {
  String key = "/_systemTime/current";
  long regCenterTime = zkRegCenter.getRegistryCenterTime(key);
  // Developer-written non-MR assertion
  assertTrue(regCenterTime<=System.currentTimeMillis());
  // LLM-generated non-MR assertion
  assertTrue(regCenterTime > 0L);
}

//Developer-written MR-based test: IF t2=getRegistryCenterTime(key)
↪  is called after t1=getRegistryCenterTime(key), THEN t1<t2
public void GetRegistryCenterTime_MRTest() {
  String key = "/_systemTime/current";
  long regCenterTime = zkRegCenter.getRegistryCenterTime(key);
  long updatedRegCenterTime = zkRegCenter.getRegistryCenterTime(key);
  assertTrue(regCenterTime < updatedRegCenterTime);
}
```

Listing 10: MR-based and non-MR-based tests[6] for `ZookeeperRegistryCenter`

tests help improve test adequacy with an increase of 10.62% and 18.91% in line coverage and mutation score, respectively. The results are in line with that reported by Xu et al.'s study [65], which observes an increase of 52.10% and 82.80% in line coverage and mutation score, respectively, by MR-based tests over EvoSuite-generated tests. Even compared with the combination of developer-written and EvoSuite-generated tests, MR-based new tests exclusively covered 113 (+6.95%) mutants and killed 88 (+7.93%) mutants. This is because, although EvoSuite can generate many inputs, it fails to generate effective test oracles that can identify incorrect outputs and solve the setups to trigger target programs. In contrast, MR-based tests combine high-quality oracles with diverse inputs and leverage developer-written setups for triggering target programs, resulting in higher test adequacy.

6 RELATED WORK

6.1 Automated Identification of MRs.

Identification of proper MRs is a key step in applying MT to specific SUTs. To efficiently identify MRs, many automated approaches have been proposed. Earlier approaches identify MRs based on a set of predefined patterns [49, 75]. Zhang et al. [73] and Zhang et al. [72] proposed search-based approaches to inferring MRs. Tsigkanos et al. [57] proposed to use LLMs to identify variable relation and input transformation in scientific software. These approaches mainly synthesize MRs for specific domains. Shin et al. [51] proposed an approach to generating executable MRs from requirements specifications using LLMs, but it still requires human effort to implement supportive functions. Ayerdi et al. [3, 4] extend Terragni et al.'s work [55] to generate MRs via genetic programming. Nolasco et al. [41] proposed MemoRIA to infer equivalence MRs between methods and method sequences. Recently, Xu et al. [65] explored a new source to automatically derive MRs. They synthesize MRs from existing test cases where domain knowledge is embedded. This served as an effective approach to reusing many encoded MRs. Such encoded MRs are prevalent, but over 70% lack an input transformation function to support reusing them on more source inputs.

To reuse these invaluable MRs, in this paper, we propose MR-Adopt to generate input transformation functions for such MRs. Integrated with the input transformations, these MRs are found helpful in enhancing test adequacy in our evaluation.

6.2 LLMs for Test Generation.

Researchers explored various LLM usages for test generation. Yuan et al. [71] studied the performance and limitations of ChatGPT in unit test generation. Xia et al. [62] built a fuzzer using LLMs as a generator of realistic test inputs and an engine for mutation. Tang et al. [54] compared the effectiveness of ChatGPT and Evosuite in unit test generation. Lemieux et al. [27] and Yang et al. [66] tried to promote the coverage of the tests generated by LLMs.

Different from these works, MR-Adopt does not use LLMs to generate tests directly. Instead, it generates the input transformation for the encoded MRs and reuses such MRs to enable more tests. In fact, using LLMs to generate correct and effective oracles and produce a large number of tests is found challenging [71]. In comparison, we reuse the human-written oracles in the encoded MRs, which are generally more reliable than LLM-generated oracles. Besides, MRs can be integrated with test input generation tools to produce abundant tests.

6.3 Enhancing LLMs for Code Generation.

LLMs are found powerful in code generation [28, 30], attracting numerous efforts to enhance the coding ability further. Some researchers designed more effective strategies of pre-training [20, 29, 33] and fine-tuning [13, 50]. Researchers also prompted LLMs with compilation messages to guide them to revise the generated code [25, 40, 71] or built a coding agent [74] to enhance LLM's code generation ability. In light of prompting with analogical reasoning [68], our work guides LLMs to generate more examples, identify the intention, and finally generate an input transformation matching the intention. Also, different from the approaches that rely purely on LLMs, we enhance the generated input transformation's quality by performing data-flow analysis to exclude irrelevant code segments from LLMs' responses and ranking the generated transformation functions based on validation with the output relation.

7 CONCLUSION

This paper presents MR-Adopt, an LLM-based approach to generate input transformations for MRs encoded in test cases that lack explicit input relations. MR-Adopt allows these encoded MRs to be reused with new source inputs, enabling the generation of new tests and achieving higher test adequacy.

Experimental results show that MR-Adopt can generate effective input transformations, where 72% input transformations are generalizable to all prepared source inputs. When integrated with these transformations and new test inputs, encoded MRs increase line coverage by 10.62% and mutation score by 18.91%, demonstrating the practical usefulness of MR-Adopt 's transformations in enhancing test adequacy.

8 DATA AVAILABILITY

We have released the code and data at https://mr-adopt.github.io/.

ACKNOWLEDGMENTS

This work is supported by the National Science Foundation of China (Grant No. 61932021), and the Hong Kong Research Grant Council/General Research Fund (Grant No. 16207120).

REFERENCES

[1] Alibaba. 2024. *codeqwen1.5.* Retrieved June 6, 2024 from https://qwenlm.github.io/blog/codeqwen1.5/

[2] Rajeev Alur, Rastislav Bodík, Garvit Juniwal, Milo M. K. Martin, Mukund Raghothaman, Sanjit A. Seshia, Rishabh Singh, Armando Solar-Lezama, Emina Torlak, and Abhishek Udupa. 2013. Syntax-guided synthesis. In *Formal Methods in Computer-Aided Design*. IEEE, 1–8.

[3] Jon Ayerdi, Valerio Terragni, Aitor Arrieta, Paolo Tonella, Goiuria Sagardui, and Maite Arratibel. 2021. Generating metamorphic relations for cyber-physical systems with genetic programming: an industrial case study. In *Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE 2021)*. 1264–1274.

[4] Jon Ayerdi, Valerio Terragni, Gunel Jahangirova, Aitor Arrieta, and Paolo Tonella. 2024. GenMorph: Automatically Generating Metamorphic Relations via Genetic Programming. *IEEE Transactions on Software Engineering* (2024), 1–12.

[5] Jialun Cao, Meiziniu Li, Yeting Li, Ming Wen, Shing-Chi Cheung, and Haiming Chen. 2022. SemMT: A Semantic-Based Testing Approach for Machine Translation Systems. *ACM Transactions on Software Engineering and Methodology* 31, 2 (2022), 34e:1–34e:36.

[6] Jialun Cao, Wuqi Zhang, and Shing-Chi Cheung. 2024. Concerned with Data Contamination? Assessing Countermeasures in Code Language Model. *CoRR* abs/2403.16898 (2024). arXiv:2403.16898

[7] Junkai Chen, Xing Hu, Zhenhao Li, Cuiyun Gao, Xin Xia, and David Lo. 2024. Code Search is All You Need? Improving Code Suggestions with Code Search. In *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering (ICSE '24)*. Association for Computing Machinery, New York, NY, USA, Article 73, 13 pages.

[8] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Pondé de Oliveira Pinto, and et al. 2021. Evaluating Large Language Models Trained on Code. *CoRR* abs/2107.03374 (2021). arXiv:2107.03374

[9] Songqiang Chen, Shuo Jin, and Xiaoyuan Xie. 2021. Testing Your Question Answering Software via Asking Recursively. In *International Conference on Automated Software Engineering*. IEEE, 104–116.

[10] Tsong Yueh Chen, Fei-Ching Kuo, Huai Liu, Pak-Lok Poon, Dave Towey, T. H. Tse, and Zhi Quan Zhou. 2018. Metamorphic Testing: A Review of Challenges and Opportunities. *Comput. Surveys* 51, 1 (2018), 4:1–4:27.

[11] DeepSeek. 2024. *deepseek-coder-7b-instruct-v1.5.* Retrieved June 6, 2024 from https://huggingface.co/deepseek-ai/deepseek-coder-7b-instruct-v1.5

[12] DeepSeek. 2024. *DeepSeek-Coder Updates.* Retrieved June 6, 2024 from https://github.com/deepseek-ai/DeepSeek-Coder/issues/89

[13] Shihan Dou, Yan Liu, Haoxiang Jia, Limao Xiong, Enyu Zhou, Wei Shen, Junjie Shan, Caishuang Huang, Xiao Wang, Xiaoran Fan, Zhiheng Xi, Yuhao Zhou, Tao Ji, Rui Zheng, Qi Zhang, Xuanjing Huang, and Tao Gui. 2024. StepCoder: Improve Code Generation with Reinforcement Learning from Compiler Feedback. *CoRR* abs/2402.01391 (2024). arXiv:2402.01391

[14] Xueying Du, Mingwei Liu, Kaixin Wang, Hanlin Wang, Junwei Liu, Yixuan Chen, Jiayi Feng, Chaofeng Sha, Xin Peng, and Yiling Lou. 2024. Evaluating Large Language Models in Class-Level Code Generation. In *nternational Conference on Software Engineering*. ACM, 81:1–81:13.

[15] Aryaz Eghbali and Michael Pradel. 2024. De-Hallucinator: Iterative Grounding for LLM-Based Code Completion. *CoRR* abs/2401.01701 (2024). arXiv:2401.01701

[16] Evalplus. 2024. *leaderboard.* Retrieved June 6, 2024 from https://evalplus.github.io/leaderboard.html

[17] Gordon Fraser and Andrea Arcuri. 2011. EvoSuite: automatic test suite generation for object-oriented software. In *Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*. ACM, 416–419.

[18] Mingyang Geng, Shangwen Wang, Dezun Dong, Haotian Wang, Ge Li, Zhi Jin, Xiaoguang Mao, and Xiangke Liao. 2024. Large Language Models are Few-Shot Summarizers: Multi-Intent Comment Generation via In-Context Learning. In *International Conference on Software Engineering*. ACM, 39:1–39:13.

[19] Sumit Gulwani. 2011. Automating string processing in spreadsheets using input-output examples. In *Symposium on Principles of Programming Languages, 2011*. ACM, 317–330.

[20] Daya Guo, Qihao Zhu, Dejian Yang, Zhenda Xie, Kai Dong, Wentao Zhang, Guanting Chen, Xiao Bi, Y. Wu, Y. K. Li, Fuli Luo, Yingfei Xiong, and Wenfeng Liang. 2024. DeepSeek-Coder: When the Large Language Model Meets Programming - The Rise of Code Intelligence. *CoRR* abs/2401.14196 (2024). arXiv:2401.14196

[21] Ari Holtzman, Jan Buys, Li Du, Maxwell Forbes, and Yejin Choi. 2020. The Curious Case of Neural Text Degeneration. In *International Conference on Learning Representations*. OpenReview.net.

[22] Kaifeng Huang, Bihuan Chen, Congying Xu, Ying Wang, Bowen Shi, Xin Peng, Yijian Wu, and Yang Liu. 2022. Characterizing usages, updates and risks of third-party libraries in Java projects. *Empirical Software Engineering* 27, 4 (2022), 90.

[23] Maliheh Izadi, Jonathan Katzy, Tim van Dam, Marc Otten, Razvan Mihai Popescu, and Arie van Deursen. 2024. Language Models for Code Completion: A Practical Evaluation. In *International Conference on Software Engineering*. ACM, 79:1–79:13.

[24] JavaParser. 2024. *JavaParser.* Retrieved June 6, 2024 from https://javaparser.org/

[25] Shuyang Jiang, Yuhao Wang, and Yu Wang. 2023. SelfEvolve: A Code Evolution Framework via Large Language Models. *CoRR* abs/2306.02907 (2023). arXiv:2306.02907

[26] Vu Le, Mehrdad Afshari, and Zhendong Su. 2014. Compiler validation via equivalence modulo inputs. In *Conference on Programming Language Design and Implementation*. ACM, 216–226.

[27] Caroline Lemieux, Jeevana Priya Inala, Shuvendu K. Lahiri, and Siddhartha Sen. 2023. CodaMosa: Escaping Coverage Plateaus in Test Generation with Pre-trained Large Language Models. In *International Conference on Software Engineering*. IEEE, 919–931.

[28] Chengshu Li, Jacky Liang, Andy Zeng, Xinyun Chen, Karol Hausman, Dorsa Sadigh, Sergey Levine, Li Fei-Fei, Fei Xia, and Brian Ichter. 2023. Chain of Code: Reasoning with a Language Model-Augmented Code Emulator. *CoRR* abs/2312.04474 (2023). arXiv:2312.04474

[29] Raymond Li, Loubna Ben Allal, Yangtian Zi, Niklas Muennighoff, Denis Kocetkov, and et al. 2023. StarCoder: may the source be with you! *CoRR* abs/2305.06161 (2023). arXiv:2305.06161

[30] Yujia Li, David H. Choi, Junyoung Chung, Nate Kushman, Julian Schrittwieser, and et al. 2022. Competition-Level Code Generation with AlphaCode. *CoRR* abs/2203.07814 (2022). arXiv:2203.07814

[31] Mikael Lindvall, Dharmalingam Ganesan, Ragnar Ardal, and Robert E. Wiegand. 2015. Metamorphic Model-Based Testing Applied on NASA DAT - An Experience Report. In *International Conference on Software Engineering*. IEEE Computer Society, 129–138.

[32] Huai Liu, Fei-Ching Kuo, Dave Towey, and Tsong Yueh Chen. 2014. How Effectively Does Metamorphic Testing Alleviate the Oracle Problem? *IEEE Transactions on Software Engineering* 40, 1 (2014), 4–22.

[33] Ziyang Luo, Can Xu, Pu Zhao, Qingfeng Sun, Xiubo Geng, Wenxiang Hu, Chongyang Tao, Jing Ma, Qingwei Lin, and Daxin Jiang. 2023. WizardCoder: Empowering Code Large Language Models with Evol-Instruct. *CoRR* abs/2306.08568 (2023). arXiv:2306.08568

[34] Haoyang Ma, Qingchao Shen, Yongqiang Tian, Junjie Chen, and Shing-Chi Cheung. 2023. Fuzzing Deep Learning Compilers with HirGen. In *International Symposium on Software Testing and Analysis*. ACM, 248–260.

[35] Lipeng Ma, Weidong Yang, Bo Xu, Sihang Jiang, Ben Fei, Jiaqing Liang, Mingjie Zhou, and Yanghua Xiao. 2024. KnowLog: Knowledge Enhanced Pre-trained Language Model for Log Understanding. In *International Conference on Software Engineering*. ACM, 32:1–32:13.

[36] Qiuyang Mang, Aoyang Fang, Boxi Yu, Hanfei Chen, and Pinjia He. 2024. Testing Graph Database Systems via Equivalent Query Rewriting. In *International Conference on Software Engineering*. ACM, 143:1–143:12.

[37] Meta. 2024. *Llama-3-8B-Instruct.* Retrieved June 6, 2024 from https://huggingface.co/meta-llama/Meta-Llama-3-8B-Instruct

[38] Meta. 2024. *Meta-Llama-3 Updates.* Retrieved June 6, 2024 from https://huggingface.co/meta-llama/Meta-Llama-3-8B-Instruct

[39] Daye Nam, Andrew Macvean, Vincent J. Hellendoorn, Bogdan Vasilescu, and Brad A. Myers. 2024. Using an LLM to Help With Code Understanding. In *International Conference on Software Engineering*. ACM, 97:1–97:13.

[40] Ansong Ni, Srini Iyer, Dragomir Radev, Veselin Stoyanov, Wen-Tau Yih, Sida I. Wang, and Xi Victoria Lin. 2023. LEVER: Learning to Verify Language-to-Code Generation with Execution. In *International Conference on Machine Learning (Proceedings of Machine Learning Research, Vol. 202)*. PMLR, 26106–26128.

[41] Agustín Nolasco, Facundo Molina, Renzo Degiovanni, Alessandra Gorla, Diego Garbervetsky, Mike Papadakis, Sebastián Uchitel, Nazareno Aguirre, and Marcelo F. Frias. 2024. Abstraction-Aware Inference of Metamorphic Relations. *Proceedings of the ACM on Software Engineering* 1, FSE (2024), 450–472.

[42] OpenAI. 2024. *GPT-3.5.* Retrieved June 6, 2024 from https://platform.openai.com/docs/models/

[43] OpenAI. 2024. *GPT-3.5 Turbo Updates.* Retrieved June 6, 2024 from https://help.openai.com/en/articles/8555514-gpt-3-5-turbo-updates

[44] Carlos Pacheco and Michael D. Ernst. 2007. Randoop: feedback-directed random testing for Java. In *Conference on Object-Oriented Programming, Systems, Languages, and Applications*. ACM, 815–816.

[45] Rangeet Pan, Vu Le, Nachiappan Nagappan, Sumit Gulwani, Shuvendu K. Lahiri, and Mike Kaufman. 2021. Can Program Synthesis be Used to Learn Merge Conflict Resolutions? An Empirical Analysis. In *IEEE/ACM International Conference on Software Engineering*. IEEE, 785–796.

[46] Pitest. 2024. *Pitest.* Retrieved June 6, 2024 from https://pitest.org/

[47] Sergio Segura, Gordon Fraser, Ana Belén Sánchez, and Antonio Ruiz Cortés. 2016. A Survey on Metamorphic Testing. *IEEE Transactions on Software Engineering* 42, 9 (2016), 805–824.

[48] Sergio Segura, José Antonio Parejo, Javier Troya, and Antonio Ruiz Cortés. 2018. Metamorphic Testing of RESTful Web APIs. *IEEE Transactions on Software Engineering* 44, 11 (2018), 1083–1099.

[49] Sergio Segura, José Antonio Parejo, Javier Troya, and Antonio Ruiz Cortés. 2018. Metamorphic testing of RESTful web APIs. In *International Conference on Software Engineering*. ACM, 882.

[50] Bo Shen, Jiaxin Zhang, Taihong Chen, Daoguang Zan, Bing Geng, An Fu, Muhan Zeng, Ailun Yu, Jichuan Ji, Jingyang Zhao, Yuenan Guo, and Qianxiang Wang. 2023. PanGu-Coder2: Boosting Large Language Models for Code with Ranking Feedback. *CoRR* abs/2307.14936 (2023). arXiv:2307.14936

[51] Seung Yeob Shin, Fabrizio Pastore, Domenico Bianculli, and Alexandra Baicoianu. 2024. Towards Generating Executable Metamorphic Relations Using Large Language Models. *CoRR* abs/2401.17019 (2024). arXiv:2401.17019

[52] Chengnian Sun, Vu Le, and Zhendong Su. 2016. Finding compiler bugs via live code mutation. In *International Conference on Object-Oriented Programming, Systems, Languages, and Applications,*. ACM, 849–863.

[53] Chang-Ai Sun, Yiqiang Liu, Zuoyi Wang, and W. K. Chan. 2016. μMT: a data mutation directed metamorphic relation acquisition methodology. In *International Workshop on Metamorphic Testing*. ACM, 12–18.

[54] Yutian Tang, Zhijie Liu, Zhichao Zhou, and Xiapu Luo. 2024. ChatGPT vs SBST: A Comparative Assessment of Unit Test Suite Generation. *IEEE Transactions on Software Engineering* (2024), 1–19.

[55] Valerio Terragni, Gunel Jahangirova, Paolo Tonella, and Mauro Pezzè. 2020. Evolutionary Improvement of Assertion Oracles. In *Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*. 1178–1189.

[56] MR-Adopt. 2024. *MR-Adopt*. Retrieved June 6, 2024 from https://mr-adopt.github.io/

[57] Christos Tsigkanos, Pooja Rani, Sebastian Müller, and Timo Kehrer. 2023. Variable Discovery with Large Language Models for Metamorphic Testing of Scientific Software. In *Computational Science - ICCS 2023 - 23rd International Conference, Prague, Czech Republic, July 3-5, 2023, Proceedings, Part I (Lecture Notes in Computer Science, Vol. 14073)*. Springer, 321–335.

[58] Ying Wang, Bihuan Chen, Kaifeng Huang, Bowen Shi, Congying Xu, Xin Peng, Yijian Wu, and Yang Liu. 2020. An Empirical Study of Usages, Updates and Risks of Third-Party Libraries in Java Projects. In *International Conference on Software Maintenance and Evolution*. IEEE, 35–45.

[59] Taylor Webb, Keith J Holyoak, and Hongjing Lu. 2023. Emergent analogical reasoning in large language models. *Nature Human Behaviour* 7, 9 (2023), 1526–1541.

[60] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed H. Chi, Quoc V. Le, and Denny Zhou. 2022. Chain-of-Thought Prompting Elicits Reasoning in Large Language Models. In *Conference on Neural Information Processing Systems*.

[61] Yi Wu, Nan Jiang, Hung Viet Pham, Thibaud Lutellier, Jordan Davis, Lin Tan, Petr Babkin, and Sameena Shah. 2023. How Effective Are Neural Networks for Fixing Security Vulnerabilities. In *International Symposium on Software Testing and Analysis*. ACM, 1282–1294.

[62] Chunqiu Steven Xia, Matteo Paltenghi, Jia Le Tian, Michael Pradel, and Lingming Zhang. 2024. Fuzz4All: Universal Fuzzing with Large Language Models. In *International Conference on Software Engineering*. ACM, 126:1–126:13.

[63] Xiaoyuan Xie, Shuo Jin, and Songqiang Chen. 2023. qaAskeR$^+$: a novel testing method for question answering software via asking recursive questions. *Automated Software Engineering* 30, 1 (2023), 14.

[64] Xiaoyuan Xie, Shuo Jin, Songqiang Chen, and Shing-Chi Cheung. 2024. Word Closure-Based Metamorphic Testing for Machine Translation. *ACM Transactions on Software Engineering and Methodology* (jul 2024).

[65] Congying Xu, Valerio Terragni, Hengcheng Zhu, Jiarong Wu, and Shing-Chi Cheung. 2024. MR-Scout: Automated Synthesis of Metamorphic Relations from Existing Test Cases. *ACM Transactions on Software Engineering and Methodology* 33, 6 (2024), 150.

[66] Chen Yang, Junjie Chen, Bin Lin, Jianyi Zhou, and Ziqi Wang. 2024. Enhancing LLM-based Test Generation for Hard-to-Cover Branches via Program Analysis. *CoRR* abs/2404.04966 (2024). arXiv:2404.04966

[67] Zhen Yang, Fang Liu, Zhongxing Yu, Jacky Wai Keung, Jia Li, Shuo Liu, Yifan Hong, Xiaoxue Ma, Zhi Jin, and Ge Li. 2024. Exploring and Unleashing the Power of Large Language Models in Automated Code Translation. *CoRR* abs/2404.14646 (2024). arXiv:2404.14646

[68] Michihiro Yasunaga, Xinyun Chen, Yujia Li, Panupong Pasupat, Jure Leskovec, Percy Liang, Ed H. Chi, and Denny Zhou. 2023. Large Language Models as Analogical Reasoners. *CoRR* abs/2310.01714 (2023). arXiv:2310.01714

[69] Yuanyuan Yuan, Shuai Wang, Mingyue Jiang, and Tsong Yueh Chen. 2021. Perception Matters: Detecting Perception Failures of VQA Models Using Metamorphic Testing. In *Conference on Computer Vision and Pattern Recognition*. Computer Vision Foundation / IEEE, 16908–16917.

[70] Zhiqiang Yuan, Mingwei Liu, Shiji Ding, Kaixin Wang, Yixuan Chen, Xin Peng, and Yiling Lou. 2024. Evaluating and improving chatgpt for unit test generation. *Proceedings of the ACM on Software Engineering* 1, FSE (2024), 1703–1726.

[71] Zhiqiang Yuan, Yiling Lou, Mingwei Liu, Shiji Ding, Kaixin Wang, Yixuan Chen, and Xin Peng. 2023. No More Manual Tests? Evaluating and Improving ChatGPT for Unit Test Generation. *CoRR* abs/2305.04207 (2023). arXiv:2305.04207

[72] Bo Zhang, Hongyu Zhang, Junjie Chen, Dan Hao, and Pablo Moscato. 2019. Automatic Discovery and Cleansing of Numerical Metamorphic Relations. In *IEEE International Conference on Software Maintenance and Evolution*. IEEE, 235–245.

[73] Jie Zhang, Junjie Chen, Dan Hao, Yingfei Xiong, Bing Xie, Lu Zhang, and Hong Mei. 2014. Search-based inference of polynomial metamorphic relations. In *ACM/IEEE International Conference on Automated Software Engineering*. ACM, 701–712.

[74] Kechi Zhang, Jia Li, Ge Li, Xianjie Shi, and Zhi Jin. 2024. CodeAgent: Enhancing Code Generation with Tool-Integrated Agent Systems for Real-World Repo-level Coding Challenges. *CoRR* abs/2401.07339 (2024). arXiv:2401.07339

[75] Zhi Quan Zhou, Liqun Sun, Tsong Yueh Chen, and Dave Towey. 2020. Metamorphic Relations for Enhancing System Understanding and Use. *IEEE Transactions on Software Engineering* 46, 10 (2020), 1120–1154.